Basics of OOPs

Abstraction

Abstraction is the concept of hiding implementation details while showing only essential features of an object. It focuses on what an object does rather than how it does it.

Key Points:

- Abstraction can be achieved using abstract classes and interfaces
- Abstract classes cannot be instantiated directly
- Abstract methods must be implemented by subclasses

Abstract Classes

```java
abstract class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    // Abstract method - must be implemented by subclasses
    public abstract void makeSound();

    // Concrete method - can be used by subclasses
    public void sleep() {
        System.out.println(name + " is sleeping");
    }
}

class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    public void makeSound() {
        System.out.println(name + " says Woof!");
    }
}
```

Interfaces

```java
interface Drawable {
    void draw(); // public and abstract by default

    default void display() { // default method (Java 8+)
        System.out.println("Displaying the drawing");
    }
}

class Circle implements Drawable {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a circle with radius: " + radius);
    }
}
```

Encapsulation

Encapsulation is the bundling of data (variables) and methods that operate on that data into a single unit (class). It also involves restricting direct access to some components using access modifiers.

Key Points:

- Use private variables to hide data
- Provide public getter and setter methods to access private data
- Helps in data security and maintenance

Example:

java

```java
class Student {
    private String name;
    private int age;
    private double gpa;

    // Constructor
    public Student(String name, int age, double gpa) {
        this.name = name;
        setAge(age); // Using setter for validation
        setGpa(gpa); // Using setter for validation
    }

    // Getter methods
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public double getGpa() {
        return gpa;
    }

    // Setter methods with validation
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name;
        }
    }

    public void setAge(int age) {
        if (age > 0 && age < 120) {
            this.age = age;
        }
    }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        }
    }
```

```java
    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age + ", GPA: " + gpa);
    }
}
```

Inheritance

Inheritance allows a class to inherit properties and methods from another class. The class that inherits is called a subclass (child class), and the class being inherited from is called a superclass (parent class).

Key Points:

- Use `extends` keyword for inheritance
- Java supports single inheritance (one parent class)
- Use `super` keyword to access parent class members
- Method overriding allows subclasses to provide specific implementations

Example:

java

```java
class Vehicle {
    protected String brand;
    protected int year;

    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    public void start() {
        System.out.println("Vehicle is starting");
    }

    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}

class Car extends Vehicle {
    private int numberOfDoors;

    public Car(String brand, int year, int numberOfDoors) {
        super(brand, year); // Call parent constructor
        this.numberOfDoors = numberOfDoors;
    }

    @Override
    public void start() {
        System.out.println("Car engine is starting with a key");
    }

    @Override
    public void displayInfo() {
        super.displayInfo(); // Call parent method
        System.out.println("Number of doors: " + numberOfDoors);
    }

    public void honk() {
        System.out.println("Car is honking: Beep! Beep!");
    }
}

class ElectricCar extends Car {
```

```java
    private int batteryCapacity;

    public ElectricCar(String brand, int year, int numberOfDoors, int batteryCapacity) {
        super(brand, year, numberOfDoors);
        this.batteryCapacity = batteryCapacity;
    }

    @Override
    public void start() {
        System.out.println("Electric car is starting silently");
    }

    public void charge() {
        System.out.println("Charging the electric car");
    }
}
```

Polymorphism

Polymorphism allows objects of different types to be treated as objects of a common base type. It enables a single interface to represent different underlying forms (data types).

Types of Polymorphism:

Method Overloading (Compile-time Polymorphism)

```java
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public String add(String a, String b) {
        return a + b;
    }
}
```

Method Overriding (Runtime Polymorphism)

java

```java
class Shape {
    public void draw() {
        System.out.println("Drawing a shape");
    }

    public double calculateArea() {
        return 0;
    }
}

class Rectangle extends Shape {
    private double length, width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a rectangle");
    }

    @Override
    public double calculateArea() {
        return length * width;
    }
}

class Triangle extends Shape {
    private double base, height;

    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a triangle");
    }

    @Override
```

```java
    public double calculateArea() {
        return 0.5 * base * height;
    }
}

// Demonstrating polymorphism
class PolymorphismDemo {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Rectangle(5, 3),
            new Triangle(4, 6),
            new Shape()
        };

        for (Shape shape : shapes) {
            shape.draw(); // Calls appropriate draw method
            System.out.println("Area: " + shape.calculateArea());
        }
    }
}
```

Quiz Questions

1. Which of the following best describes abstraction in Java? a) Hiding implementation details while showing essential features b) Creating multiple methods with the same name c) Inheriting properties from a parent class d) Bundling data and methods together

Answer: a) Hiding implementation details while showing essential features

2. What access modifier should be used for variables in encapsulation to ensure data hiding? a) public b) protected c) private d) default

Answer: c) private

3. Which keyword is used to inherit from a class in Java? a) implements b) extends c) inherits d) super

Answer: b) extends

4. Method overloading is an example of which type of polymorphism? a) Runtime polymorphism b) Compile-time polymorphism c) Dynamic polymorphism d) Interface polymorphism

Answer: b) Compile-time polymorphism

5. What happens when you try to instantiate an abstract class directly in Java? a) It creates an object successfully b) It gives a compilation error c) It creates a null object d) It throws a runtime exception

Answer: b) It gives a compilation error

Practice Coding Questions

1. Create an abstract class `Appliance` with:

- Protected fields: `brand` and `power`
- A constructor to initialize these fields
- An abstract method `operate()`
- A concrete method `displaySpecs()` that prints the brand and power

Then create a concrete class `WashingMachine` that extends `Appliance` and implements the `operate()` method.

2. Design a class `BankAccount` following encapsulation principles:

- Private fields: `accountNumber`, `balance`, `accountHolderName`
- Public methods to get account details (getters)
- Public methods to deposit and withdraw money with appropriate validation
- A method to display account information

3. Create a class hierarchy:

- Base class `Employee` with fields `name`, `id`, and method `calculateSalary()`
- Derived class `FullTimeEmployee` with additional field `monthlySalary`
- Derived class `PartTimeEmployee` with fields `hoursWorked` and `hourlyRate`
- Override `calculateSalary()` method in both derived classes appropriately

4. Demonstrate method overloading by creating a class `MathOperations` with multiple `multiply` methods:

- `multiply(int a, int b)`
- `multiply(double a, double b)`
- `multiply(int a, int b, int c)`
- Create a main method to test all overloaded methods

**5. Create an interface `Playable` with methods `play()`, `pause()`, and `stop()`. Implement this interface in classes `MusicPlayer` and `VideoPlayer`, providing different implementations for each method.